

Programação Concorrente e Paralela

Primeira Prova - 30/09/2024

1. Escreva uma solução para o problema do barbeiro dorminhoco usando troca de mensagens síncrona. (10/3 pontos)

Como a comunicação é síncrona é possível fazer as interações entre cliente e barbeiro diretamente pelas primitivas de comunicação. Assim, todos os clientes fazem envios para os canais usados pelo barbeiro, que recebe as mensagens em FIFO (como o canal é definido). Sendo comunicação síncrona, os clientes apenas fazem a chamada para o canal *chair* depois que o barbeiro receber a mensagem que enviaram no canal *barb*, garantindo que apenas um cliente sente na cadeira por vez. Depois de fazer o corte, o barbeiro precisa enviar a mensagem de término ao cliente correto, por isso o vetor de canais, e apenas chama o próximo cliente depois que o cliente atual receber essa mensagem.

```
chan barb, cli[n], chair;

process Barber {
  int who;
  while (true) {
    receive barb(who);    // Espera por um cliente
    receive chair();      // Espera que o cliente sente na cadeira
    cut_hair();
    synch_send cli[who](); // Avisa que já terminou o corte e espera ele sair
  }
}

process Client [i= 0 to N] {
  synch_send barb(i);    // Avisa o barbeiro que ele chegou e espera ser chamado
  synch_send chair();    // Senta na cadeira e espera terminar o corte
  receive cli[i]();      // Ao ser avisado vai embora
}
```

2. A solução a seguir supostamente faz a implementação de troca de valores entre N processos parceiros, usando troca de mensagens assíncrona, para determinar o menor e o maior desses valores. Verifique se a solução é correta. (10/3 pontos)

```
chan values[n](int);
int v[n];    // assuma que v é inicializado em algum momento

process P[i=0 to n-1] {
  int new, smallest=v[i], largest=v[i];
  for [j=0 to n-1 st j!=i]
    send values[j](v[i]);
  for [j=1 to n-1] {
    receive values[i](new);
    if (new < smallest)
      smallest = new;
    if (new > largest)
      largest = new;
  }
}
```

A análise do código deve ser feita considerando os dois laços *for* existentes. Então temos:

No primeiro laço cada processo envia $n-1$ mensagens para todos os demais processos. Essas mensagens serão enfileiradas nos respectivos canais.

Como as mensagens estão enfileiradas, então no segundo laço cada processo receberá o valor vindo de cada um dos demais processos, fazendo a identificação do maior e do menor valor entre todos encontrados.

Assim, ao final dos dois laços teremos identificados os valores desejados. Portanto a solução está correta.

3. Escreva uma solução para o problema de múltiplos produtores-consumidores, com buffer de N posições, usando monitores. (10/3 pontos)

Aqui temos K produtores e M consumidores, usando um buffer de N posições para passagem de dados. O que temos que fazer é evitar que um produtor produza algum item se não tivermos posição disponível e que um consumidor não consuma se não houver item a ser consumido, controlando ainda os índices da última posição ocupada e da última posição consumida. Teremos duas variáveis condicionais, uma para segurar produtores e outra para segurar consumidores.

```
monitor factory {
    cond prod, cons;
    sem mutex;
    int lastIn=0, lastOut=0, nprod=0;
    data buffer[N];

    procedure producer [i=0 to K] {
        data item;

        while (true) {
            if (nprod == N) // bloqueia produtor se buffer está cheio
                wait (prod);
            produz(item);
            buffer[lastIn] = item;
            P(mutex); // Garante a não interferência sobre valores dos contadores
            nprod++; lastIn = (lastIn++) % N;
            V(mutex);
            if (! empty(cons)) signal(cons); // eventualmente libera algum consumidor
        }
    }

    procedure consumer [j=0 to M] {
        data recebido;

        while (true) {
            if (nprod == 0) // bloqueia consumidor se buffer está vazio
                wait (cons);
            recebido = buffer[lastOut];
            P(mutex); // Garante a não interferência sobre valores dos contadores
            nprod--; lastOut = (lastOut++) % N;
            V(mutex);
            if (! empty(prod)) signal(prod); // eventualmente libera algum produtor
        }
    }
}
```

4. Considere que um conjunto de processos coopere através de semáforos distribuídos, ou seja, um tipo de semáforo em que seja permitido o bloqueio de um processo mesmo não usando memória compartilhada. Implemente um servidor de semáforos usando RPC para cumprir essa função. (10/3 pontos)

```

module Semaphore
    op Pdistr (int);
    op Vdistr (int);
    op SemInit (int) returns int;

body
    sem sems[N];
    int Semaph[N], nextsem=0;

    proc SemInit (int val) returns sempos {
        Semaph[nextsem] = val;
        sempos = nextsem; // retorna o índice do vetor ocupado pelo semáforo
        nextsem++;
    }

    proc Pdistr(int s) {
        if (Semaph[s] > 0)
            Semaph[s]--;
        else
            P(sems[s]); // Não retorna para o processo chamador, bloqueando-o
                        // indiretamente ao ter a operação bloqueada por sem[s]
    }

    proc Vdistr(int s) {
        if (Semaph[s] == 0)
            V(sem[s]);
        Semaph[s]++;
    }
end Semaphore

```

Deve ficar claro que omitimos um detalhe (nem tão pequeno), que é o de que processos compartilhando um semáforo devem ter um mecanismo para que cada processo receba o índice daquele semáforo nos vetores `Semaph` e `sem`. Esse mecanismo pode envolver a modificação na função `InitSem`, que receberia dois parâmetros, o valor inicial e o nome, devendo buscar nos vetores se aquele nome já existe ou não, retornando o índice de acordo com o resultado dessa busca.

OBSERVAÇÃO - Caso respondam as 4 questões será descartada a questão 1, mesmo que alguma das outras tenha apenas algumas poucas linhas. **Apenas três questões serão corrigidas em qualquer hipótese.**

Manual...

1. Troca de Mensagens:

- Declaração de canal: *chan chan_name (data_types);*
- Envio de mensagens assíncrono: *send chan_name(data);*
- Envio de mensagens síncrono: *sync_send chan_name(data);*
- Recebimento de mensagens: *receive chan_name(data);*
- Verificação de canal: *empty(chan_name); // true if chan_name is empty*

2. Monitores:

- Sintaxe -

```
monitor mname {  
    declarations of permanent variables  
    initialization code  
    procedures  
}
```

- Ferramentas auxiliares -

--Variáveis condicionais:

cond cv; // declara uma fila definida pelo apontador cv

--Uso de cond:

empty (cv); // verdade se cv é uma fila vazia

wait (cv); // coloca processo na fila de cv

signal (cv); // acorda processo esperando por cv (FIFO)

signal_all (cv); // acorda todos processos esperando por cv

wait (cv, rank); // insere na fila cv, na posição dada por rank

minrank (cv); // rank do processo na frente da fila de cv

3. RPC:

```
module mname  
    headers das operacoes exportadas  
body  
    variable declarations  
    initialization code  
    procedures for exported operations  
    local procedures and processes  
end mname
```

declaração de operação

```
op opname (formals) [ returns result ]
```

```
proc opname (formal identifiers) returns result_ident  
    local declarations  
    statements  
end;
```